MyBatis 进阶

MyBatis的高级映射及延迟加载

准备工作

准备数据库表

一个班级对应多个学生。

MySQL密码：20060329

数据库：mybatis

班级表：t_clazz

学生表：t_student

```sql
-- 创建数据库（如果不存在的话）并使用该数据库
CREATE DATABASE IF NOT EXISTS mybatis DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE mybatis;

-- 如果表已经存在则删除，方便重复执行测试
DROP TABLE IF EXISTS t_student;
DROP TABLE IF EXISTS t_clazz;

-- 1. 创建班级表 t_clazz
CREATE TABLE t_clazz (
  cid INT PRIMARY KEY COMMENT '班级编号',
  cname VARCHAR(255) COMMENT '班级名称'
) COMMENT '班级表';

-- 2. 创建学生表 t_student
CREATE TABLE t_student (
  sid INT PRIMARY KEY AUTO_INCREMENT COMMENT '学生编号',
  sname VARCHAR(255) COMMENT '学生姓名',
  cid INT COMMENT '班级编号（外键）'
) COMMENT '学生表';

-- 3. 向班级表中插入数据
INSERT INTO t_clazz (cid, cname) VALUES
(1001, '高三1班'),
(1002, '高三2班');

-- 4. 向学生表中插入数据
INSERT INTO t_student (sid, sname, cid) VALUES
(1, '张三', 1001),
(2, '李四', 1001),
(3, '王五', 1001),
(4, '赵六', 1002),
(5, '钱七', 1002);
```

对象 t_clazz @mybatis (本地MySQL) - 表

表配置文件 开始事务 单元格编辑器

cid # int	cname varchar(255)
1001	高三1班
1002	高三2班

表配置文件 开始事务 单元格编辑器

sid # int	sname varchar(255)	cid # int
1	张三	1001
2	李四	1001
3	王五	1001
4	赵六	1002
5	钱七	1002

字段 索引 外键 检查 触发器 选项 注释 SQL 预览

名称	类型	长度	小数点	不是 null	虚拟	键	注释
cid	int			☑	☐	🔑 1	班级编号
cname	varchar	255		☐	☐		班级名称

保存 添加字段 插入字段 删除字段 主键 上移 下移

字段 索引 外键 检查 触发器 选项 注释 SQL 预览

名称	类型	长度	小数点	不是 null	虚拟	键	注释
sid	int			☑	☐	🔑 1	学生编号
sname	varchar	255		☐	☐		学生姓名
cid	int			☐	☐		班级编号（外键）

准备配置文件

模块名：mybatis-009-advanced-mapping

打包方式：jar

依赖：mybatis依赖--3.5.19、mysql驱动依赖--8.4.0、junit依赖--4.13.2、logback依赖--1.5.6

jdk：JDK21

- 配置pom.xml 文件

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>lzt.mybatis</groupId>
    <artifactId>my_mybatis-08-select</artifactId>
    <version>1.0-SNAPSHOT</version>

    <packaging>jar</packaging>
```

```xml
    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.mybatis</groupId>
            <artifactId>mybatis</artifactId>
            <version>3.5.19</version>
        </dependency>

        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>8.4.0</version>
        </dependency>

        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>ch.qos.logback</groupId>
            <artifactId>logback-classic</artifactId>
            <version>1.5.6</version>
        </dependency>
    </dependencies>

    <build>
        <resources>
            <resource>
                <directory>src/main/java</directory>
                <includes>
                    <include>**/*.xml</include>
                </includes>
            </resource>
            <resource>
                <directory>src/main/resources</directory>
            </resource>
        </resources>
    </build>
</project>
```

- 配置文件：mybatis-config.xml、logback.xml、jdbc.properties

```xml
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
        <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
            <pattern>[%thread] %-5level %logger{50} - %msg%n</pattern>
        </encoder>
    </appender>

    <logger name="lzt.mybatis.mapper" level="DEBUG"/>

    <root level="INFO">
        <appender-ref ref="STDOUT" />
```

```
    </root>
</configuration>
```

```
jdbc.driver=com.mysql.cj.jdbc.Driver
jdbc.url=jdbc:mysql://localhost:3306/mybatis?useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai
jdbc.username=root
jdbc.password=20060329
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

    <properties resource="jdbc.properties"/>

    <settings>
        <setting name="mapUnderscoreToCamelCase" value="true"/>
    </settings>

    <typeAliases>
        <package name="lzt.mybatis.pojo"/>
    </typeAliases>

    <environments default="development">
        <environment id="development">
            <transactionManager type="JDBC"/>
            <dataSource type="POOLED">
                <property name="driver" value="${jdbc.driver}"/>
                <property name="url" value="${jdbc.url}"/>
                <property name="username" value="${jdbc.username}"/>
                <property name="password" value="${jdbc.password}"/>
            </dataSource>
        </environment>
    </environments>

    <mappers>
        <package name="lzt.mybatis.mapper"/>
    </mappers>

</configuration>
```

- 拷贝工具类：SqlSessionUtil.java

```
package lzt.mybatis.utils;

import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;

import java.io.IOException;

/**
 * MyBatis 的 SqlSession 工具类
 */
public class SqlSessionUtil {

    // 工具类的构造方法一般私有化，防止别人 new 对象
    private SqlSessionUtil() {}
```

```java
    private static SqlSessionFactory sqlSessionFactory;

    // 全局唯一的一个 ThreadLocal 对象，用于绑定当前线程和 SqlSession
    private static ThreadLocal<SqlSession> local = new ThreadLocal<>();

    // 静态代码块，类加载时只执行一次，保证全局只有一个 SqlSessionFactory
    static {
        try {
            sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-
config.xml"));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    /**
     * 获取 SqlSession 对象
     * @return SqlSession
     */
    public static SqlSession openSession() {
        SqlSession sqlSession = local.get();
        if (sqlSession == null) {
            // 如果当前线程没有绑定 SqlSession，就新建一个，并绑定到当前线程
            sqlSession = sqlSessionFactory.openSession();
            local.set(sqlSession);
        }
        return sqlSession;
    }

    /**
     * 关闭 SqlSession 对象，并从当前线程中移除
     */
    public static void close(SqlSession sqlSession) {
        if (sqlSession != null) {
            sqlSession.close();
            // 务必解绑
            local.remove();
        }
    }
}
```

- 创建pojo：Student、Clazz
- Student.java

```java
package lzt.mybatis.pojo;

public class Student {
    private Integer sid;
    private String sname;
    // 多对一：一个学生属于一个班级，这里体现为关联一个 Clazz 对象
    private Clazz clazz;

    public Student() {
    }

    public Student(Integer sid, String sname, Clazz clazz) {
        this.sid = sid;
        this.sname = sname;
        this.clazz = clazz;
    }

    public Integer getSid() {
```

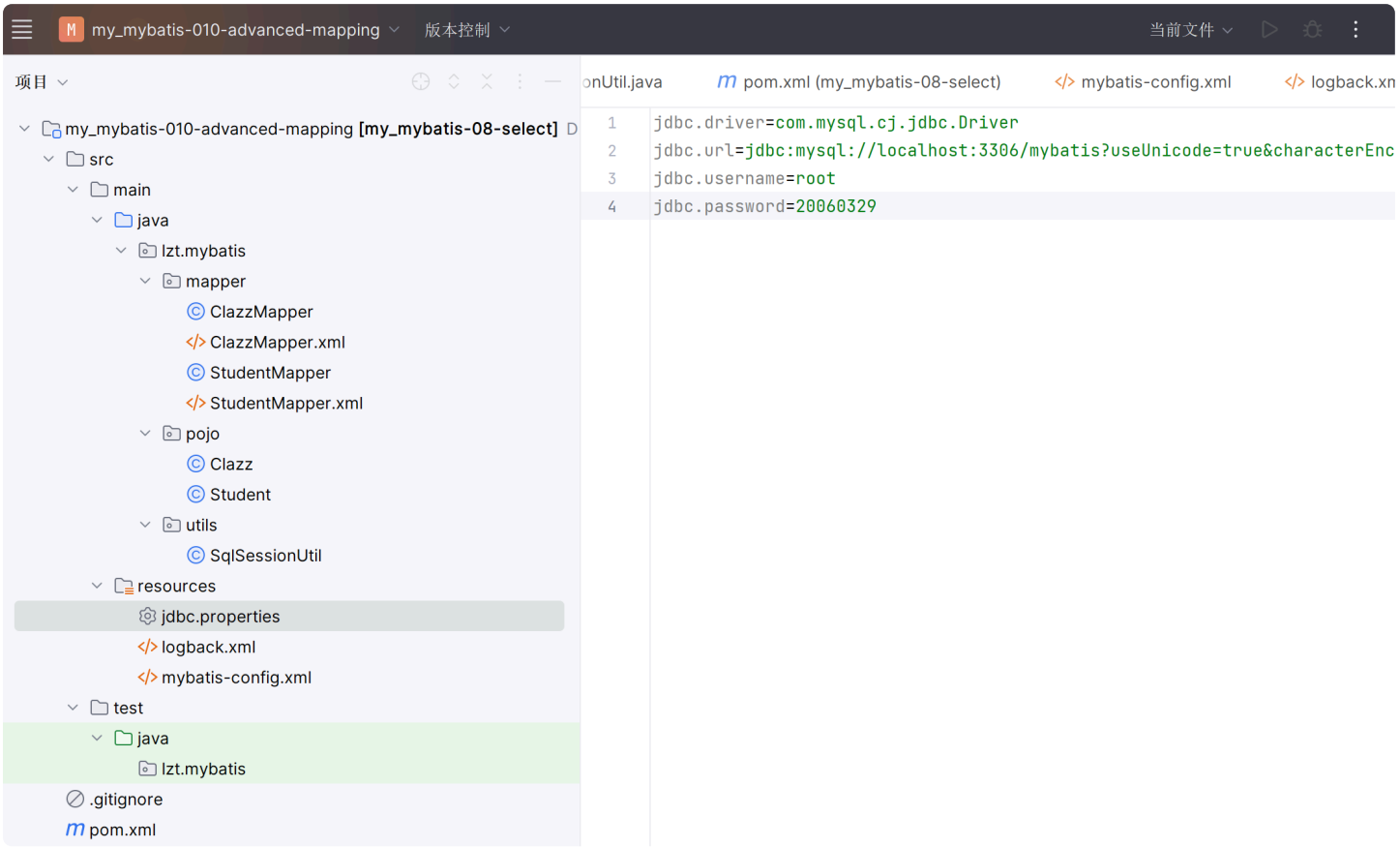
```java
        return sid;
    }

    public void setSid(Integer sid) {
        this.sid = sid;
    }

    public String getSname() {
        return sname;
    }

    public void setSname(String sname) {
        this.sname = sname;
    }

    public Clazz getClazz() {
        return clazz;
    }

    public void setClazz(Clazz clazz) {
        this.clazz = clazz;
    }

    @Override
    public String toString() {
        return "Student{" +
                "sid=" + sid +
                ", sname='" + sname + '\'' +
                ", clazz=" + clazz +
                '}';
    }
}
```

- Clazz

```java
package lzt.mybatis.pojo;

import java.util.List;

public class Clazz {
    private Integer cid;
    private String cname;

    // 一对多：一个班级有多个学生，这里体现为关联一个 Student 的 List 集合
    private List<Student> stus;

    public Clazz() {
    }

    public Clazz(Integer cid, String cname, List<Student> stus) {
        this.cid = cid;
        this.cname = cname;
        this.stus = stus;
    }

    public Integer getCid() {
        return cid;
    }

    public void setCid(Integer cid) {
        this.cid = cid;
    }
```

```java
    public String getCname() {
        return cname;
    }

    public void setCname(String cname) {
        this.cname = cname;
    }

    public List<Student> getStus() {
        return stus;
    }

    public void setStus(List<Student> stus) {
        this.stus = stus;
    }

    @Override
    public String toString() {
        return "Clazz{" +
                "cid=" + cid +
                ", cname='" + cname + '\'' +
                ", stus=" + stus +
                '}';
    }
}
```

创建mapper接口：StudentMapper、ClazzMapper
创建mapper映射文件：StudentMapper.xml、ClazzMapper.xml



多对一

多种方式，常见的包括三种：

- 第一种方式：一条SQL语句，级联属性映射。

- 第二种方式：一条SQL语句，association。
- 第三种方式：两条SQL语句，分步查询。（这种方式常用：优点一是可复用。优点二是支持懒加载。）

第一种方式：级联属性映射

pojo类Student中添加一个属性：Clazz clazz; 表示学生关联的班级对象。

```
package lzt.mybatis.pojo;

public class Student {
    private Integer sid;
    private String sname;
    // 多对一：一个学生属于一个班级，这里体现为关联一个 Clazz 对象
    private Clazz clazz;

    public Student() {
    }

    public Student(Integer sid, String sname, Clazz clazz) {
        this.sid = sid;
        this.sname = sname;
        this.clazz = clazz;
    }

    public Integer getSid() {
        return sid;
    }

    public void setSid(Integer sid) {
        this.sid = sid;
    }

    public String getSname() {
        return sname;
    }

    public void setSname(String sname) {
        this.sname = sname;
    }

    public Clazz getClazz() {
        return clazz;
    }

    public void setClazz(Clazz clazz) {
        this.clazz = clazz;
    }

    @Override
    public String toString() {
        return "Student{" +
                "sid=" + sid +
                ", sname='" + sname + '\'' +
                ", clazz=" + clazz +
                '}';
    }
}
```

- StudentMapper.java

```java
package lzt.mybatis.mapper;

import lzt.mybatis.pojo.Student;

public interface StudentMapper {
    /**
     * 根据学生 id 查询学生信息，并同时查询出其所属的班级信息
     * @param sid 学生编号
     * @return 包含班级信息的学生对象
     */
    Student selectBySid(Integer sid);
}
```

- StudentMapper.xml

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
        PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.StudentMapper">

    <resultMap id="studentResultMap" type="Student">
        <id property="sid" column="sid"/>
        <result property="sname" column="sname"/>
        <result property="clazz.cid" column="cid"/>
        <result property="clazz.cname" column="cname"/>
    </resultMap>

    <select id="selectBySid" resultMap="studentResultMap">
        select s.*, c.* from t_student s left join t_clazz c on s.cid = c.cid where s.sid = #{sid}
    </select>

</mapper>
```

- StudentMapperTest.java

```java
package lzt.mybatis;

import lzt.mybatis.mapper.StudentMapper;
import lzt.mybatis.pojo.Student;
import lzt.mybatis.utils.SqlSessionUtil;
import org.junit.Test;

public class StudentMapperTest {

    @Test
    public void testSelectBySid(){
        // 获取 Mapper 接口的代理对象
        StudentMapper mapper = SqlSessionUtil.openSession().getMapper(StudentMapper.class);
        // 执行查询，查询 sid 为 1 的学生（张三）
        Student student = mapper.selectBySid(1);
        // 打印结果，你应该能看到张三的信息以及他所属的高三1班的信息
        System.out.println(student);
    }
}
```

执行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==>  Preparing: select s.*, c.* from t_student s left join t_clazz c on
 s.cid = c.cid where s.sid = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==> Parameters: 1(Integer)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - <==      Total: 1
Student{sid=1, sname='张三', clazz=Clazz{cid=1001, cname='高三1班', stus=null}}
```

第二种方式：association

其他位置都不需要修改，只需要修改resultMap中的配置：association即可。

- StudentMapper.xml

```xml
<resultMap id="studentResultMap" type="Student">
  <id property="sid" column="sid"/>
  <result property="sname" column="sname"/>
  <association property="clazz" javaType="Clazz">
    <id property="cid" column="cid"/>
    <result property="cname" column="cname"/>
  </association>
</resultMap>
```

association翻译为：关联。

学生对象关联一个班级对象。

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==>  Preparing: select s.*, c.* from t_student s left join t_clazz c on
 s.cid = c.cid where s.sid = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==> Parameters: 1(Integer)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - <==      Total: 1
Student{sid=1, sname='张三', clazz=Clazz{cid=1001, cname='高三1班', stus=null}}
```

第三种方式：分步查询

其他位置不需要修改，只需要修改以及添加以下三处：

第一处：association中select位置填写sqlId。sqlId=namespace+id。其中column属性作为这条子sql语句的条件。

- StudentMapper.xml

```xml
    <resultMap id="studentResultMap" type="Student">
        <id property="sid" column="sid"/>
        <result property="sname" column="sname"/>
        <association property="clazz"
                     select="lzt.mybatis.mapper.ClazzMapper.selectByCid"
                     column="cid"/>
    </resultMap>

    <select id="selectBySid" resultMap="studentResultMap">
        select s.* from t_student s where sid = #{sid}
    </select>
```

> 注意：`<select id="selectBySid" resultMap="studentResultMap">`也修改了SQL语句

- 第二处：在ClazzMapper接口中添加方法

```java
package lzt.mybatis.mapper;
```

```java
import lzt.mybatis.pojo.Clazz;

/**
 * Clazz映射器接口
 */
public interface ClazzMapper {

    /**
     * 根据cid获取Clazz信息
     * @param cid
     * @return
     */
    Clazz selectByCid(Integer cid);
}
```

- 第三处：在ClazzMapper.xml文件中进行配置

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
        PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.ClazzMapper">

    <select id="selectByCid" resultType="Clazz">
        select cid, cname from t_clazz where cid = #{cid}
    </select>

</mapper>
```

运行之前的 StudentMapperTest.testSelectBySid 测试方法

执行结果，可以很明显看到先后有两条sql语句执行：

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==> Preparing: select s.* from t_student s where sid = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==> Parameters: 1(Integer)
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - ====> Preparing: select cid, cname from t_clazz where cid = ?
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - ====> Parameters: 1001(Integer)
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - <====      Total: 1
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - <==      Total: 1
Student{sid=1, sname='张三', clazz=Clazz{cid=1001, cname='高三1班', stus=null}}
```

分步优点：

- 第一个优点：代码复用性增强。
- 第二个优点：支持延迟加载。【暂时访问不到的数据可以先不查询。提高程序的执行效率。】

多对一延迟加载

要想支持延迟加载，非常简单，只需要在association标签中添加fetchType="lazy"即可。

修改StudentMapper.xml文件：

```
<resultMap id="studentResultMap" type="Student">
    <id property="sid" column="sid"/>
    <result property="sname" column="sname"/>
    <association property="clazz"
                 select="lzt.mybatis.mapper.ClazzMapper.selectByCid"
                 column="cid"
                 fetchType="lazy"/>
</resultMap>
```

我们现在只查询学生名字，修改测试程序：

```
    @Test
    public void testSelectBySid(){
        // 获取 Mapper 接口的代理对象
        StudentMapper mapper = SqlSessionUtil.openSession().getMapper(StudentMapper.class);
        // 执行查询，查询 sid 为 1 的学生（张三）
        Student student = mapper.selectBySid(1);
        // 打印结果，你应该能看到张三的信息以及他所属的高三1班的信息
//        System.out.println(student);

        String sname = student.getSname();
        System.out.println("学生姓名：" + sname);
    }
```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==>  Preparing: select s.* from t_student s where sid = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==> Parameters: 1(Integer)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - <==      Total: 1
学生姓名：张三
```

如果后续需要使用到学生所在班级的名称，这个时候才会执行关联的sql语句，修改测试程序：

```
    @Test
    public void testSelectBySid(){
        // 获取 Mapper 接口的代理对象
        StudentMapper mapper = SqlSessionUtil.openSession().getMapper(StudentMapper.class);
        // 执行查询，查询 sid 为 1 的学生（张三）
        Student student = mapper.selectBySid(1);
        // 打印结果，你应该能看到张三的信息以及他所属的高三1班的信息
//        System.out.println(student);
        // 只获取学生名字
        String sname = student.getSname();
        System.out.println("学生姓名：" + sname);

        // 到这里之后，想获取班级名字了
        String cname = student.getClazz().getCname();
        System.out.println("学生的班级名称：" + cname);
    }
```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==>  Preparing: select s.* from t_student s where sid = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==> Parameters: 1(Integer)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - <==      Total: 1
学生姓名：张三
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - ==> Preparing: select cid, cname from t_clazz where cid = ?
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - ==> Parameters: 1001(Integer)
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - <==      Total: 1
学生的班级名称：高三1班
```

通过以上的执行结果可以看到，只有当使用到班级名称之后，才会执行关联的sql语句，这就是延迟加载。

在mybatis中如何开启全局的延迟加载呢？需要setting配置，如下：

设置（settings）

这是 MyBatis 中极为重要的调整设置，它们会改变 MyBatis 的运行时行为。 下表描述了设置中各项设置的含义、默认值等。

设置名	描述	有效值	默认值
cacheEnabled	全局性地开启或关闭所有映射器配置文件中已配置的任何缓存。	true | false	true
lazyLoadingEnabled	延迟加载的全局开关。当开启时，所有关联对象都会延迟加载。 特定关联关系中可通过设置 `fetchType` 属性来覆盖该项的开关状态。	true | false	false
aggressiveLazyLoading	开启时，任一方法的调用都会加载该对象的所有延迟加载属性。 否则，每个延迟加载属性会按需加载（参考 `lazyLoadTriggerMethods`）。	true | false	false （在 3.4.1 及之前的版本中默认为 true）

老杜

- mybatis-config.xml

```
<settings>
  <setting name="lazyLoadingEnabled" value="true"/>
</settings>
```

- **把fetchType="lazy"去掉。**
  - 修改StudentMapper.xml文件

```
    <resultMap id="studentResultMap" type="Student">
        <id property="sid" column="sid"/>
        <result property="sname" column="sname"/>
        <association property="clazz"
                     select="lzt.mybatis.mapper.ClazzMapper.selectByCid"
                     column="cid"/>
    </resultMap>
```

执行以下程序：

```
    @Test
    public void testSelectBySid(){
        // 获取 Mapper 接口的代理对象
        StudentMapper mapper = SqlSessionUtil.openSession().getMapper(StudentMapper.class);
        // 执行查询，查询 sid 为 1 的学生（张三）
        Student student = mapper.selectBySid(1);
        // 打印结果，你应该能看到张三的信息以及他所属的高三1班的信息
//        System.out.println(student);
        // 只获取学生名字
        String sname = student.getSname();
        System.out.println("学生姓名：" + sname);

        // 到这里之后，想获取班级名字了
        String cname = student.getClazz().getCname();
        System.out.println("学生的班级名称：" + cname);
```

```
    }
```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==>  Preparing: select s.* from t_student s where sid = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==> Parameters: 1(Integer)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - <==      Total: 1
学生姓名：张三
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - ==>  Preparing: select cid, cname from t_clazz where cid = ?
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - ==> Parameters: 1001(Integer)
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - <==      Total: 1
学生的班级名称：高三1班
```

通过以上的测试可以看出，我们已经开启了全局延迟加载策略。

开启全局延迟加载之后，所有的sql都会支持延迟加载，如果某个sql你不希望它支持延迟加载怎么办呢？将fetchType设置为eager：

- 修改StudentMapper.xml文件

```
    <resultMap id="studentResultMap" type="Student">
        <id property="sid" column="sid"/>
        <result property="sname" column="sname"/>
        <association property="clazz"
                     select="lzt.mybatis.mapper.ClazzMapper.selectByCid"
                     column="cid"
                     fetchType="eager"/>
    </resultMap>
```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==>  Preparing: select s.* from t_student s where sid = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - ==> Parameters: 1(Integer)
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - ====>  Preparing: select cid, cname from t_clazz where cid = ?
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - ====> Parameters: 1001(Integer)
[main] DEBUG lzt.mybatis.mapper.ClazzMapper.selectByCid - <====      Total: 1
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySid - <==      Total: 1
学生姓名：张三
学生的班级名称：高三1班
```

这样的话，针对某个特定的sql，你就关闭了延迟加载机制。

后期我们要不要开启延迟加载机制，主要看实际的业务需求是怎样的。

一对多

一对多的实现，通常是在一的一方中有List集合属性。

在Clazz类中添加 `List<Student> stus;` 属性。

- Clazz.java

```
package lzt.mybatis.pojo;

import java.util.List;

public class Clazz {
    private Integer cid;
    private String cname;

    // 一对多：一个班级有多个学生，这里体现为关联一个 Student 的 List 集合
    private List<Student> stus;
```

```java
    public Clazz() {
    }

    public Clazz(Integer cid, String cname, List<Student> stus) {
        this.cid = cid;
        this.cname = cname;
        this.stus = stus;
    }

    public Integer getCid() {
        return cid;
    }

    public void setCid(Integer cid) {
        this.cid = cid;
    }

    public String getCname() {
        return cname;
    }

    public void setCname(String cname) {
        this.cname = cname;
    }

    public List<Student> getStus() {
        return stus;
    }

    public void setStus(List<Student> stus) {
        this.stus = stus;
    }

    @Override
    public String toString() {
        return "Clazz{" +
                "cid=" + cid +
                ", cname='" + cname + '\'' +
                ", stus=" + stus +
                '}';
    }
}
```

一对多的实现通常包括两种实现方式：

- 第一种方式：collection
- 第二种方式：分步查询

第一种方式：collection

- ClazzMapper接口

```java
package lzt.mybatis.mapper;
import lzt.mybatis.pojo.Clazz;

/**
 * Clazz映射器接口
 */
public interface ClazzMapper {

    /**
     * 根据cid获取Clazz信息
```

```java
     * @param cid
     * @return
     */
    Clazz selectByCid(Integer cid);

    /**
     * 根据班级编号查询班级信息。同时班级中所有的学生信息也要查询。
     * @param cid
     * @return
     */
    Clazz selectClazzAndStusByCid(Integer cid);
}
```

- ClazzMapper.xml

```xml
<resultMap id="clazzResultMap" type="Clazz">
  <id property="cid" column="cid"/>
  <result property="cname" column="cname"/>
  <collection property="stus" ofType="Student">
    <id property="sid" column="sid"/>
    <result property="sname" column="sname"/>
  </collection>
</resultMap>

<select id="selectClazzAndStusByCid" resultMap="clazzResultMap">
  select * from t_clazz c join t_student s on c.cid = s.cid where c.cid = #{cid}
</select>
```

> 注意是ofType，表示“集合中的类型”。

- ClazzMapperTest.testSelectClazzAndStusByCid

```java
package lzt.mybatis;

import lzt.mybatis.mapper.ClazzMapper;
import lzt.mybatis.pojo.Clazz;
import lzt.mybatis.utils.SqlSessionUtil;
import org.junit.Test;

public class ClazzMapperTest {
    @Test
    public void testSelectClazzAndStusByCid() {
        ClazzMapper mapper = SqlSessionUtil.openSession().getMapper(ClazzMapper.class);
        Clazz clazz = mapper.selectClazzAndStusByCid(1001);
        System.out.println(clazz);
    }
}
```

执行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG l.m.mapper.ClazzMapper.selectClazzAndStusByCid - ==>  Preparing: select * from t_clazz c join t_student s on c.cid =
 s.cid where c.cid = ?
[main] DEBUG l.m.mapper.ClazzMapper.selectClazzAndStusByCid - ==> Parameters: 1001(Integer)
[main] DEBUG l.m.mapper.ClazzMapper.selectClazzAndStusByCid - <==      Total: 3
Clazz{cid=1001, cname='高三1班', stus=[Student{sid=1, sname='张三', clazz=null}, Student{sid=2, sname='李四', clazz=null},
 Student{sid=3, sname='王五', clazz=null}]}
```

第二种方式：分步查询

修改以下三个位置即可：

- ClazzMapper.xml

```
    <resultMap id="clazzResultMap" type="Clazz">
        <id property="cid" column="cid"/>
        <result property="cname" column="cname"/>
        <!--主要看这里-->
        <collection property="stus"
                    select="lzt.mybatis.mapper.StudentMapper.selectByCid"
                    column="cid"/>
    </resultMap>

    <!--sql语句也变化了-->
    <select id="selectClazzAndStusByCid" resultMap="clazzResultMap">
        select * from t_clazz c where c.cid = #{cid}
    </select>
```

- StudentMapper接口

```
import java.util.List;

/**
* 根据班级编号获取所有的学生。
* @param cid
* @return
*/
List<Student> selectByCid(Integer cid);
```

- StudentMapper.xml

```
<select id="selectByCid" resultType="Student">
  select * from t_student where cid = #{cid}
</select>
```

执行下面代码：

```
    @Test
    public void testSelectClazzAndStusByCid() {
        ClazzMapper mapper = SqlSessionUtil.openSession().getMapper(ClazzMapper.class);
        Clazz clazz = mapper.selectClazzAndStusByCid(1001);
        System.out.println(clazz);
    }
```

执行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG l.m.mapper.ClazzMapper.selectClazzAndStusByCid - ==>  Preparing: select * from t_clazz c where c.cid = ?
[main] DEBUG l.m.mapper.ClazzMapper.selectClazzAndStusByCid - ==> Parameters: 1001(Integer)
[main] DEBUG l.m.mapper.ClazzMapper.selectClazzAndStusByCid - <==      Total: 1
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByCid - ==>  Preparing: select * from t_student where cid = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByCid - ==> Parameters: 1001(Integer)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByCid - <==      Total: 3
Clazz{cid=1001, cname='高三1班', stus=[Student{sid=1, sname='张三', clazz=null}, Student{sid=2, sname='李四', clazz=null},
 Student{sid=3, sname='王五', clazz=null}]}
```

一对多延迟加载

一对多延迟加载机制和多对一是一样的。同样是通过两种方式：

- 第一种：fetchType="lazy"
- 第二种：修改全局的配置setting，**lazyLoadingEnabled=true，**如果开启全局延迟加载，想让某个sql不使用延迟加载：fetchType="eager"

> 和多对一延迟加载步骤一模一样，修改ClazzMapper.xml文件和测试 ClazzMapperTest 类文件即可

MyBatis的缓存

缓存：cache

缓存的作用：通过减少IO的方式，来提高程序的执行效率。

mybatis的缓存：将select语句的查询结果放到缓存（内存）当中，下一次还是这条select语句的话，直接从缓存中取，不再查数据库。一方面是减少了IO。另一方面不再执行繁琐的查找算法。效率大大提升。

mybatis缓存包括：

- 一级缓存：将查询到的数据存储到SqlSession中。
- 二级缓存：将查询到的数据存储到SqlSessionFactory中。
- 或者集成其它第三方的缓存：比如EhCache【Java语言开发的】、Memcache【C语言开发的】等。

缓存只针对于DQL语句，也就是说缓存机制只对应select语句。

一级缓存

一级缓存默认是开启的。不需要做任何配置。

原理：只要使用同一个SqlSession对象执行同一条SQL语句，就会走缓存。

项目名称：my_mybatis-11-cache

- CarMapper接口

```
package lzt.mybatis.mapper;
import lzt.mybatis.pojo.Car;

public interface CarMapper {

    /**
     * 根据id获取Car信息。
     * @param id
     * @return
     */
    Car selectById(Long id);
}
```

- CarMapper.xml

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
        PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.CarMapper">

    <select id="selectById" resultType="Car">
        select * from t_car where id = #{id}
    </select>

</mapper>
```

- CarMapperTest.testSelectById

```java
package lzt.mybatis;

import lzt.mybatis.mapper.CarMapper;
import lzt.mybatis.pojo.Car;
import lzt.mybatis.utils.SqlSessionUtil;
import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;
import org.junit.Test;

public class CarMapperTest {

    @Test
    public void testSelectById() throws Exception{
        // 注意：不能使用我们封装的SqlSessionUtil工具类。
        SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
        SqlSessionFactory sqlSessionFactory = builder.build(Resources.getResourceAsStream("mybatis-config.xml"));

        SqlSession sqlSession1 = sqlSessionFactory.openSession();

        CarMapper mapper1 = sqlSession1.getMapper(CarMapper.class);
        Car car1 = mapper1.selectById(5L);
        System.out.println(car1);

        CarMapper mapper2 = sqlSession1.getMapper(CarMapper.class);
        Car car2 = mapper2.selectById(5L);
        System.out.println(car2);

        SqlSession sqlSession2 = sqlSessionFactory.openSession();

        CarMapper mapper3 = sqlSession2.getMapper(CarMapper.class);
        Car car3 = mapper3.selectById(5L);
        System.out.println(car3);

        CarMapper mapper4 = sqlSession2.getMapper(CarMapper.class);
        Car car4 = mapper4.selectById(5L);
        System.out.println(car4);

    }
}
```

执行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==>  Preparing: select * from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Parameters: 5(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - <==      Total: 1
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==>  Preparing: select * from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Parameters: 5(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - <==      Total: 1
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
```

一级缓存

什么情况下不走缓存?

- 第一种：不同的SqlSession对象。
- 第二种：查询条件变化了。

一级缓存失效情况包括两种：

- 第一种：第一次查询和第二次查询之间，手动清空了一级缓存。
  - 清空一级缓存

```
sqlSession.clearCache();
```

`sqlSession.clearCache();` 是一个在使用 MyBatis（一种 Java 持久化框架）时可以调用的方法。这个方法的作用是清空 MyBatis 中的缓存。

在使用 MyBatis 进行数据库操作时，为了提高性能，MyBatis 会将一些查询结果缓存起来，以便在后续的操作中能够快速访问。但有时候我们希望清空这些缓存，以便重新加载最新的数据或者释放内存。这时就可以使用 `sqlSession.clearCache();` 方法来手动清空缓存。

简而言之，这个方法就像是一个清理工具，用来清理掉已经缓存的查询结果，以便在需要的时候重新查询数据库。

- 第二种：第一次查询和第二次查询之间，执行了增删改操作。【这个增删改和哪张表没有关系，只要有insert delete update操作，一级缓存就失效。】
- CarMapper接口

```
/**
* 保存账户信息
*/
void insertAccount();
```

- CarMapper.xml

```
<insert id="insertAccount">
    insert into powernode.t_act values(3, 'act003', 10000)
</insert>
```

针对 CarMapperTest.testSelectById 代码，增加 mapper1.insertAccount(); 代码，如下：

```
    @Test
    public void testSelectById() throws Exception{
        // 注意：不能使用我们封装的SqlSessionUtil工具类。
        SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
        SqlSessionFactory sqlSessionFactory = builder.build(Resources.getResourceAsStream("mybatis-config.xml"));
```

```java
        SqlSession sqlSession1 = sqlSessionFactory.openSession();

        CarMapper mapper1 = sqlSession1.getMapper(CarMapper.class);
        Car car1 = mapper1.selectById(5L);
        System.out.println(car1);

        mapper1.insertAccount();

        CarMapper mapper2 = sqlSession1.getMapper(CarMapper.class);
        Car car2 = mapper2.selectById(5L);
        System.out.println(car2);

        SqlSession sqlSession2 = sqlSessionFactory.openSession();

        CarMapper mapper3 = sqlSession2.getMapper(CarMapper.class);
        Car car3 = mapper3.selectById(5L);
        System.out.println(car3);

        CarMapper mapper4 = sqlSession2.getMapper(CarMapper.class);
        Car car4 = mapper4.selectById(5L);
        System.out.println(car4);
    }
```

执行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Preparing: select * from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Parameters: 5(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - <==      Total: 1
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
[main] DEBUG lzt.mybatis.mapper.CarMapper.insertAccount - ==> Preparing: insert into powernode.t_act values(3, 'act003', 10000)
[main] DEBUG lzt.mybatis.mapper.CarMapper.insertAccount - ==> Parameters:
[main] DEBUG lzt.mybatis.mapper.CarMapper.insertAccount - <==    Updates: 1
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Preparing: select * from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Parameters: 5(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - <==      Total: 1
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Preparing: select * from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Parameters: 5(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - <==      Total: 1
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
```

二级缓存

二级缓存的范围是SqlSessionFactory。

使用二级缓存需要具备以下几个条件：

1. `<setting name="cacheEnabled" value="true">` 全局性地开启或关闭所有映射器配置文件中已配置的任何缓存。默认就是true，无需设置。
2. 在需要使用二级缓存的CarMapper.xml文件中添加配置：`<cache />`
3. 使用二级缓存的实体类对象必须是可序列化的，也就是必须实现java.io.Serializable接口
4. SqlSession对象关闭或提交之后，一级缓存中的数据才会被写入到二级缓存当中。此时二级缓存才可用。

测试二级缓存：

- CarMapper.xml
  - 开启当前 Mapper 的二级缓存
  - 在 `<mapper>` 标签内部添加 `<cache/>` 标签即可。

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
        PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.CarMapper">

    <cache/>

    <select id="selectById" resultType="Car">
        select * from t_car where id = #{id}
    </select>

    <insert id="insertAccount">
        insert into t_act values(3, 'act003', 10000)
    </insert>

</mapper>
```

> 在 `mybatis-config.xml` 中 `<setting name="cacheEnabled" value="true"/>` 默认就是开启的，所以 `mybatis-config.xml` 可以保持原样，不需要修改。

- 修改 Car.java

```java
package lzt.mybatis.pojo;
import java.io.Serializable; //第一处修改：导包
/**
 * 封装汽车信息的 POJO 类
 * 对应数据库表：t_car
 */
//第二处修改：增加 implements Serializable 连接起来
public class Car implements Serializable {

    private static final long serialVersionUID = 1L; //第三处修改
    private Long id;
    private String carNum;
    private String brand;
    private Double guidePrice;
    private String produceTime;
    private String carType;
    // ... 下方的构造方法、Getter/Setter、toString 保持不变 ...
}
```

- CarMapperTest.java

```java
    @Test
    public void testSelectById2() throws Exception {
        SqlSessionFactory sqlSessionFactory = new
SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-config.xml"));

        // 开启第一个 SqlSession
        SqlSession sqlSession1 = sqlSessionFactory.openSession();
        CarMapper mapper1 = sqlSession1.getMapper(CarMapper.class);
        Car car1 = mapper1.selectById(5L);
        System.out.println(car1);

        // 关键一步：关闭 sqlSession1，此时数据才会被写入二级缓存
        sqlSession1.close();
```

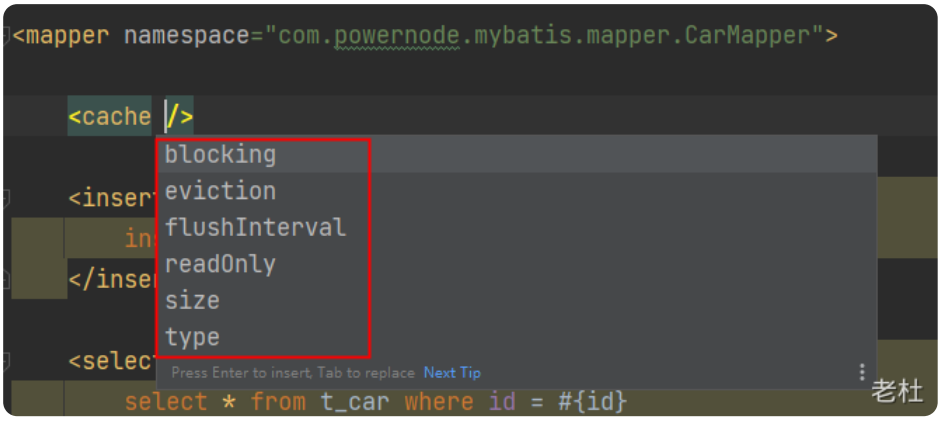
```java
        // 开启第二个 SqlSession
        SqlSession sqlSession2 = sqlSessionFactory.openSession();
        CarMapper mapper2 = sqlSession2.getMapper(CarMapper.class);
        // 再次查询同一个 ID
        Car car2 = mapper2.selectById(5L);
        System.out.println(car2);

    }
```

- 执行结果

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper - Cache Hit Ratio [lzt.mybatis.mapper.CarMapper]: 0.0
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Preparing: select * from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Parameters: 5(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - <==      Total: 1
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
[main] WARN  org.apache.ibatis.io.SerialFilterChecker - As you are using functionality that deserializes object streams, it is
 recommended to define the JEP-290 serial filter. Please refer to https://docs.oracle
 .com/pls/topic/lookup?ctx=javase15&id=GUID-8296D8E8-2B93-4B9A-856E-0A65AF9B8C66
[main] DEBUG lzt.mybatis.mapper.CarMapper - Cache Hit Ratio [lzt.mybatis.mapper.CarMapper]: 0.5
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
```

二级缓存的失效：只要两次查询之间出现了增删改操作。二级缓存就会失效。【一级缓存也会失效】

二级缓存的相关配置：



1. eviction：指定从缓存中移除某个对象的淘汰算法。默认采用LRU策略。
   a. LRU：Least Recently Used。最近最少使用。优先淘汰在间隔时间内使用频率最低的对象。(其实还有一种淘汰算法LFU，最不常用。)
   b. FIFO：First In First Out。一种先进先出的数据缓存器。先进入二级缓存的对象最先被淘汰。
   c. SOFT：软引用。淘汰软引用指向的对象。具体算法和JVM的垃圾回收算法有关。
   d. WEAK：弱引用。淘汰弱引用指向的对象。具体算法和JVM的垃圾回收算法有关。
2. flushInterval：
   a. 二级缓存的刷新时间间隔。单位毫秒。如果没有设置。就代表不刷新缓存，只要内存足够大，一直会向二级缓存中缓存数据。除非执行了增删改。
3. readOnly：
   a. true：多条相同的sql语句执行之后返回的对象是共享的同一个。性能好。但是多线程并发可能会存在安全问题。
   b. false：多条相同的sql语句执行之后返回的对象是副本，调用了clone方法。性能一般。但安全。
4. size：
   a. 设置二级缓存中最多可存储的java对象数量。默认值1024。

MyBatis集成EhCache

集成EhCache是为了代替mybatis自带的二级缓存。一级缓存是无法替代的。

mybatis对外提供了接口，也可以集成第三方的缓存组件。比如EhCache、Memcache等。都可以。

EhCache是Java写的。Memcache是C语言写的。所以mybatis集成EhCache较为常见，按照以下步骤操作，就可以完成集成：

- 第一步：引入mybatis整合ehcache的依赖。
  - pom.xml

```
<!--mybatis集成ehcache的组件-->
<dependency>
  <groupId>org.mybatis.caches</groupId>
  <artifactId>mybatis-ehcache</artifactId>
  <version>1.2.2</version>
</dependency>
```

- 第二步：在类的根路径下新建echcache.xml文件，并提供以下配置信息

```
<?xml version="1.0" encoding="UTF-8"?>
<ehcache xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:noNamespaceSchemaLocation="http://ehcache.org/ehcache.xsd"
         updateCheck="false">
    <!--磁盘存储:将缓存中暂时不使用的对象,转移到硬盘,类似于Windows系统的虚拟内存-->
    <diskStore path="e:/ehcache"/>

    <!--defaultCache: 默认的管理策略-->
    <!--eternal: 设定缓存的elements是否永远不过期。如果为true，则缓存的数据始终有效，如果为false那么还要根据
timeToIdleSeconds，timeToLiveSeconds判断-->
    <!--maxElementsInMemory: 在内存中缓存的element的最大数目-->
    <!--overflowToDisk: 如果内存中数据超过内存限制，是否要缓存到磁盘上-->
    <!--diskPersistent: 是否在磁盘上持久化。指重启jvm后，数据是否有效。默认为false-->
    <!--timeToIdleSeconds: 对象空闲时间(单位：秒)，指对象在多长时间没有被访问就会失效。只对eternal为false的有效。默认值0，表
示一直可以访问-->
    <!--timeToLiveSeconds: 对象存活时间(单位：秒)，指对象从创建到失效所需要的时间。只对eternal为false的有效。默认值0，表示一
直可以访问-->
    <!--memoryStoreEvictionPolicy: 缓存的3 种清空策略-->
    <!--FIFO: first in first out (先进先出)-->
    <!--LFU: Less Frequently Used (最少使用).意思是一直以来最少被使用的。缓存的元素有一个hit 属性，hit 值最小的将会被清出缓
存-->
    <!--LRU: Least Recently Used(最近最少使用). (ehcache 默认值).缓存的元素有一个时间戳，当缓存容量满了，而又需要腾出地方来缓
存新的元素的时候，那么现有缓存元素中时间戳离当前时间最远的元素将被清出缓存-->
    <defaultCache eternal="false" maxElementsInMemory="1000" overflowToDisk="false" diskPersistent="false"
                  timeToIdleSeconds="0" timeToLiveSeconds="600" memoryStoreEvictionPolicy="LRU"/>

</ehcache>
```

- 第三步：修改CarMapper.xml文件中的 `<cache/>` 标签，添加type属性。

```
<cache type="org.mybatis.caches.ehcache.EhcacheCache"/>
```

第四步：编写测试程序使用。

```
    @Test
    public void testSelectById2() throws Exception {
        SqlSessionFactory sqlSessionFactory = new
SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-config.xml"));

        // 开启第一个 SqlSession
        SqlSession sqlSession1 = sqlSessionFactory.openSession();
        CarMapper mapper1 = sqlSession1.getMapper(CarMapper.class);
```

```java
        Car car1 = mapper1.selectById(5L);
        System.out.println(car1);

        // 关键一步：关闭 sqlSession1，此时数据才会被写入二级缓存
        sqlSession1.close();

        // 开启第二个 SqlSession
        SqlSession sqlSession2 = sqlSessionFactory.openSession();
        CarMapper mapper2 = sqlSession2.getMapper(CarMapper.class);
        // 再次查询同一个 ID
        Car car2 = mapper2.selectById(5L);
        System.out.println(car2);
    }
```

```
D:\Java21\bin\java.exe ...
[main] WARN  net.sf.ehcache.config.ConfigurationFactory - No configuration found. Configuring ehcache from ehcache-failsafe.xml
 found in the classpath: jar:file:/C:/Users/jack/.m2/repository/net/sf/ehcache/ehcache/2.10.9.2/ehcache-2.10.9.2
 .jar!/ehcache-failsafe.xml
[main] DEBUG lzt.mybatis.mapper.CarMapper - Cache Hit Ratio [lzt.mybatis.mapper.CarMapper]: 0.0
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==>  Preparing: select * from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Parameters: 5(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - <==      Total: 1
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
[main] DEBUG lzt.mybatis.mapper.CarMapper - Cache Hit Ratio [lzt.mybatis.mapper.CarMapper]: 0.5
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}

进程已结束，退出代码为 0
```

MyBatis的逆向工程

所谓的逆向工程是：根据数据库表逆向生成Java的pojo类，SqlMapper.xml文件，以及Mapper接口类等。

要完成这个工作，需要借助别人写好的逆向工程插件。

思考：使用这个插件的话，需要给这个插件配置哪些信息？

- pojo类名、包名以及生成位置。
- SqlMapper.xml文件名以及生成位置。
- Mapper接口名以及生成位置。
- 连接数据库的信息。
- 指定哪些表参与逆向工程。
- ……

逆向工程配置与生成

第一步：基础环境准备

新建模块：my_mybatis-12-generator

打包方式：jar

第二步：在pom中添加逆向工程插件

- pom.xml

```pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <!-- 项目基本信息 -->
    <groupId>lzt.mybatis</groupId>
    <artifactId>my_mybatis-12-generator</artifactId>
    <version>1.0-SNAPSHOT</version>

    <!-- 构建配置 -->
    <build>
        <plugins>
            <!-- MyBatis Generator 插件配置 -->
            <plugin>
                <groupId>org.mybatis.generator</groupId>
                <artifactId>mybatis-generator-maven-plugin</artifactId>
                <version>1.4.1</version>
                <configuration>
                    <!-- 允许覆盖已存在的文件 -->
                    <overwrite>true</overwrite>
                </configuration>
                <dependencies>
                    <!-- 添加MySQL驱动依赖，使插件能连接数据库 -->
                    <dependency>
                        <groupId>mysql</groupId>
                        <artifactId>mysql-connector-java</artifactId>
                        <version>8.0.30</version>
                    </dependency>
                </dependencies>
            </plugin>
        </plugins>
    </build>
</project>
```

第三步：配置generatorConfig.xml

该文件名必须叫做：generatorConfig.xml

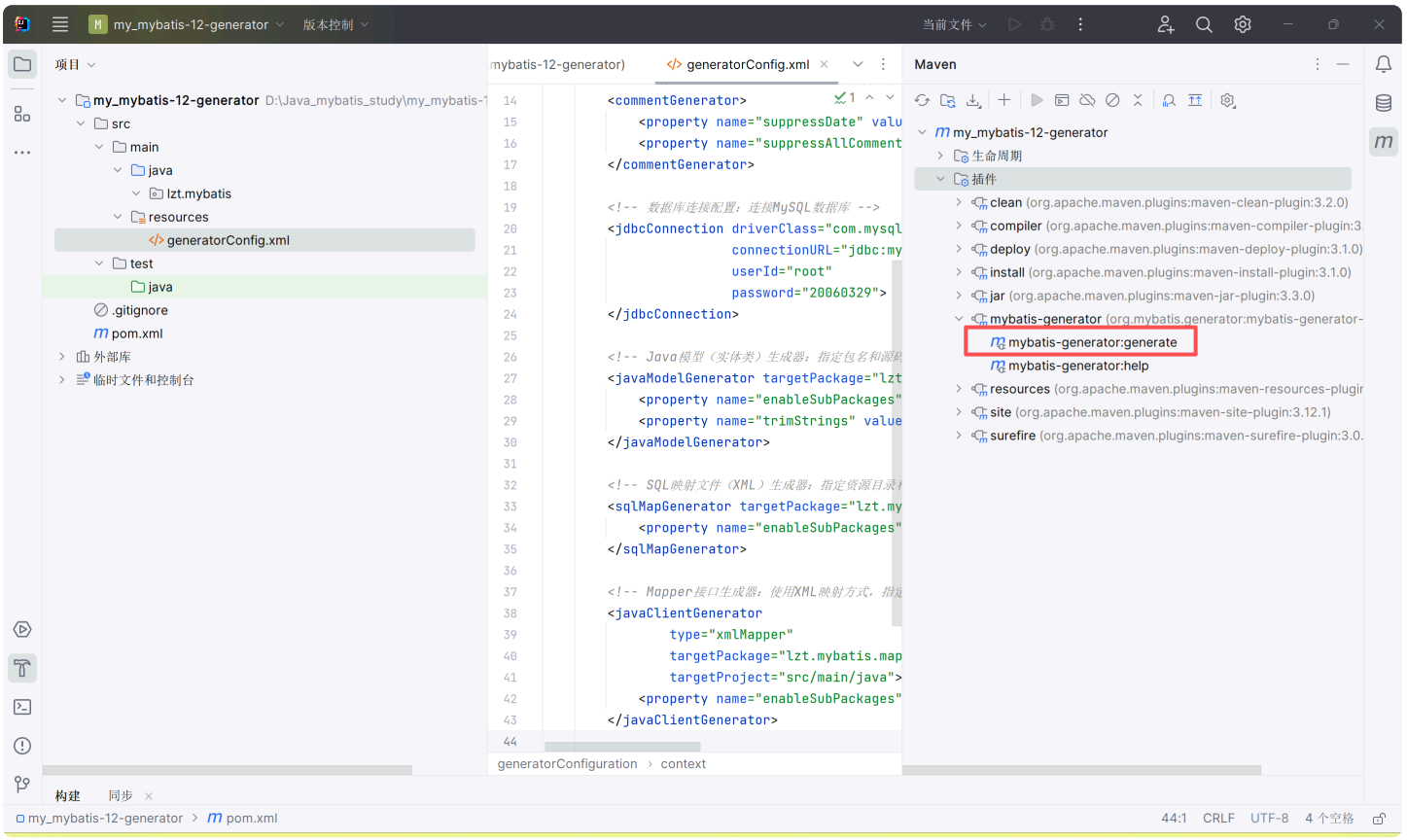
该文件必须放在类的根路径下。

- generatorConfig.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE generatorConfiguration
        PUBLIC "-//mybatis.org//DTD MyBatis Generator Configuration 1.0//EN"
        "http://mybatis.org/dtd/mybatis-generator-config_1_0.dtd">

<!-- MyBatis Generator 配置文件 -->
<generatorConfiguration>
    <!-- 上下文配置，targetRuntime="MyBatis3" 表示生成MyBatis3兼容的代码 -->
    <context id="DB2Tables" targetRuntime="MyBatis3">
        <!-- 插件：防止生成的XML文件被重复合并（避免覆盖手写代码） -->
        <plugin type="org.mybatis.generator.plugins.UnmergeableXmlMappersPlugin"/>

        <!-- 注释生成器配置：这里设置为不生成任何注释（suppressAllComments=true）也不生成日期注释 -->
        <commentGenerator>
            <property name="suppressDate" value="true"/>
            <property name="suppressAllComments" value="true"/>
        </commentGenerator>

        <!-- 数据库连接配置：连接MySQL数据库 -->
        <jdbcConnection driverClass="com.mysql.cj.jdbc.Driver"
                        connectionURL="jdbc:mysql://localhost:3306/powernode"
                        userId="root"
```

```
                    password="root">
        </jdbcConnection>

        <!-- Java模型（实体类）生成器：指定包名和源码目录 -->
        <javaModelGenerator targetPackage="lzt.mybatis.pojo" targetProject="src/main/java">
            <property name="enableSubPackages" value="true"/> <!-- 根据表名启用子包 -->
            <property name="trimStrings" value="true"/>       <!-- 对字符串属性调用trim() -->
        </javaModelGenerator>

        <!-- SQL映射文件（XML）生成器：指定资源目录和包名 -->
        <sqlMapGenerator targetPackage="lzt.mybatis.mapper" targetProject="src/main/resources">
            <property name="enableSubPackages" value="true"/>
        </sqlMapGenerator>

        <!-- Mapper接口生成器：使用XML映射方式，指定接口包名和源码目录 -->
        <javaClientGenerator
                type="xmlMapper"
                targetPackage="lzt.mybatis.mapper"
                targetProject="src/main/java">
            <property name="enableSubPackages" value="true"/>
        </javaClientGenerator>

        <!-- 数据库表映射：表名 t_car，生成的实体类名 Car -->
        <table tableName="t_car" domainObjectName="Car"/>

    </context>
</generatorConfiguration>
```

第四步：运行插件



运行后：

测试逆向工程生成的是否好用

第一步：环境准备

- 依赖：mybatis依赖、mysql驱动依赖、junit依赖、logback依赖
- jdbc.properties
- mybatis-config.xml
- logback.xml

第二步：编写测试程序

- GeneratorTest.testGenerator

```
package com.powernode.mybatis.test;

import com.powernode.mybatis.mapper.CarMapper;
import com.powernode.mybatis.pojo.Car;
import com.powernode.mybatis.pojo.CarExample;
import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;
import org.junit.Test;

import java.math.BigDecimal;
import java.util.List;

public class GeneratorTest {
    @Test
    public void testGenerator() throws Exception{
        SqlSessionFactory sqlSessionFactory = new
SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-config.xml"));
        SqlSession sqlSession = sqlSessionFactory.openSession();
        CarMapper mapper = sqlSession.getMapper(CarMapper.class);
        // 增
        /*Car car = new Car();
        car.setCarNum("1111");
        car.setBrand("比亚迪唐");
        car.setGuidePrice(new BigDecimal(30.0));
        car.setProduceTime("2010-10-12");
        car.setCarType("燃油车");
        int count = mapper.insert(car);
```

```
        System.out.println("插入了几条记录: " + count);*/
        // 删
        /*int count = mapper.deleteByPrimaryKey(83L);
        System.out.println("删除了几条记录: " + count);*/
        // 改
        // 根据主键修改
        /*Car car = new Car();
        car.setId(89L);
        car.setGuidePrice(new BigDecimal(20.0));
        car.setCarType("新能源");
        int count = mapper.updateByPrimaryKey(car);
        System.out.println("更新了几条记录: " + count);*/
        // 根据主键选择性修改
        /*car = new Car();
        car.setId(89L);
        car.setCarNum("3333");
        car.setBrand("宝马520Li");
        car.setProduceTime("1999-01-10");
        count = mapper.updateByPrimaryKeySelective(car);
        System.out.println("更新了几条记录: " + count);*/

        // 查一个
        Car car = mapper.selectByPrimaryKey(89L);
        System.out.println(car);
        // 查所有
        List<Car> cars = mapper.selectByExample(null);
        cars.forEach(c -> System.out.println(c));
        // 多条件查询
        // QBC 风格: Query By Criteria 一种查询方式, 比较面向对象, 看不到sql语句。
        CarExample carExample = new CarExample();
        carExample.createCriteria()
                .andBrandEqualTo("丰田霸道")
                .andGuidePriceGreaterThan(new BigDecimal(60.0));
        carExample.or().andProduceTimeBetween("2000-10-11", "2022-10-11");

        mapper.selectByExample(carExample);
        sqlSession.commit();
    }
}
```

MyBatis使用PageHelper

limit分页

mysql的limit后面两个数字：

- 第一个数字：startIndex（起始下标。下标从0开始。）
- 第二个数字：pageSize（每页显示的记录条数）

假设已知页码pageNum，还有每页显示的记录条数pageSize，第一个数字可以动态的获取吗？

- startIndex = (pageNum - 1) * pageSize

所以，标准通用的mysql分页SQL：

```
select
  *
from
  tableName ......
limit
  (pageNum - 1) * pageSize, pageSize
```

使用mybatis应该怎么做?

借用之前项目名称为：my_mybatis-08-select 为实现模块

- CarMapper接口

```
   import java.util.List;
   import org.apache.ibatis.annotations.Param;
/**
    * 通过分页的方式获取Car列表
    * @param startIndex 页码
    * @param pageSize 每页显示记录条数
    * @return
    */
    List<Car> selectAllByPage(@Param("startIndex") Integer startIndex, @Param("pageSize") Integer pageSize);
```

- CarMapper.xml

```
    <select id="selectAllByPage" resultType="Car">
        select * from t_car limit #{startIndex},#{pageSize}
    </select>
```

- PageTest.testPage

```
package lzt.mybatis;

import lzt.mybatis.mapper.CarMapper;
import lzt.mybatis.pojo.Car;
import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;
import org.junit.Test;

import java.util.List;

public class PageTest {
    @Test
    public void testPage()throws Exception{
        SqlSessionFactory sqlSessionFactory = new
SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-config.xml"));
        SqlSession sqlSession = sqlSessionFactory.openSession();
        CarMapper mapper = sqlSession.getMapper(CarMapper.class);

        // 页码
        Integer pageNum = 2;
        // 每页显示记录条数
        Integer pageSize = 3;
        // 起始下标
        Integer startIndex = (pageNum - 1) * pageSize;

        List<Car> cars = mapper.selectAllByPage(startIndex, pageSize);
```

```
        cars.forEach(car -> System.out.println(car));

        sqlSession.commit();
        sqlSession.close();
    }
}
```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllByPage - ==>  Preparing: select * from t_car limit ?,?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllByPage - ==> Parameters: 3(Integer), 3(Integer)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllByPage - <==      Total: 3
Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=30, carNum='2001', brand='兰博基尼', guidePrice=100.0, produceTime='1998-10-11', carType='燃油车'}
```

获取数据不难，难的是获取分页相关的数据比较难。可以借助mybatis的PageHelper插件。

PageHelper插件

使用PageHelper插件进行分页，更加的便捷。

第一步：引入依赖

- pom.xml

```
<dependency>
  <groupId>com.github.pagehelper</groupId>
  <artifactId>pagehelper</artifactId>
  <version>5.3.1</version>
</dependency>
```

第二步：在mybatis-config.xml文件中配置插件

typeAliases标签下面进行配置：

```
<plugins>
  <plugin interceptor="com.github.pagehelper.PageInterceptor"></plugin>
</plugins>
```

第三步：编写Java代码

- CarMapper接口

```
List<Car> selectAll();
```

- CarMapper.xml

```
<select id="selectAll" resultType="Car">
  select * from t_car
</select>
```

关键点：

- 在查询语句之前开启分页功能。
- 在查询语句之后封装PageInfo对象。（PageInfo对象将来会存储到request域当中。在页面上展示。）
- PageTest.testPageHelper

```
import com.github.pagehelper.PageHelper;
import com.github.pagehelper.PageInfo;  //导包

@Test
public void testPageHelper() throws Exception{
    SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-
config.xml"));
    SqlSession sqlSession = sqlSessionFactory.openSession();
    CarMapper mapper = sqlSession.getMapper(CarMapper.class);

    // 开启分页
    PageHelper.startPage(2, 2);

    // 执行查询语句
    List<Car> cars = mapper.selectAll();

    // 获取分页信息对象
    PageInfo<Car> pageInfo = new PageInfo<>(cars, 5);

    System.out.println(pageInfo);
}
```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll_COUNT - ==>  Preparing: SELECT count(0) FROM t_car
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll_COUNT - ==> Parameters:
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll_COUNT - <==      Total: 1
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll - ==>  Preparing: select * from t_car LIMIT ?, ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll - ==> Parameters: 2(Long), 2(Integer)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll - <==      Total: 2
PageInfo{pageNum=2, pageSize=2, size=2, startRow=3, endRow=4, total=8, pages=4, list=Page{count=true, pageNum=2, pageSize=2,
 startRow=2, endRow=4, total=8, pages=4, reasonable=false, pageSizeZero=false}[Car{id=3, carNum='102', brand='丰田未来', guidePrice=40
 .3, produceTime='2014-10-05', carType='氢能源'}, Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05',
 carType='氢能源'}], prePage=1, nextPage=3, isFirstPage=false, isLastPage=false, hasPreviousPage=true, hasNextPage=true,
 navigatePages=5, navigateFirstPage=1, navigateLastPage=4, navigatepageNums=[1, 2, 3, 4]}
```

对执行结果进行格式化：

```
PageInfo{
  pageNum=2, pageSize=2, size=2, startRow=3, endRow=4, total=8, pages=4,
  list=Page{count=true, pageNum=2, pageSize=2, startRow=2, endRow=4, total=8, pages=4, reasonable=false,
pageSizeZero=false}
  [Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'},
  Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}],
  prePage=1, nextPage=3, isFirstPage=false, isLastPage=false, hasPreviousPage=true, hasNextPage=true,
  navigatePages=5, navigateFirstPage=1, navigateLastPage=4, navigatepageNums=[1, 2, 3, 4]
}
```

MyBatis的注解式开发

mybatis中也提供了注解式开发方式，采用注解可以减少Sql映射文件的配置。

当然，使用注解式开发的话，sql语句是写在java程序中的，这种方式也会给sql语句的维护带来成本。

官方是这么说的：

> 使用注解来映射简单语句会使代码显得更加简洁，但对于稍微复杂一点的语句，Java 注解不仅力不从心，还会让你本就复杂的 SQL 语句更加混乱不堪。 因此，如果你需要做一些很复杂的操作，最好用 XML 来映射语句。

使用注解编写复杂的SQL是这样的：

```
@Update("<script> update table_name set grade='三年级' " +
            " <if test=\"name != null\"> , name = #{name} </if> " +
            " <if test=\"sex != null\"> ,  sex = #{sex}</if>" +
            " where num = #{num}</script>")
    void update(Student student);
```

老杜

> 原则：简单sql可以注解。复杂sql使用xml。

还是借用之前项目名称为：my_mybatis-08-select 为实现模块

@Insert

- CarMapper接口

```
import org.apache.ibatis.annotations.Insert;

public interface CarMapper {

    @Insert(value="insert into t_car values(null,#{carNum},#{brand},#{guidePrice},#{produceTime},#{carType})")
    int insert(Car car);
}
```

- AnnotationTest.testInsert

```
package lzt.mybatis;
import lzt.mybatis.mapper.CarMapper;
import lzt.mybatis.pojo.Car;
import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;
import org.junit.Test;

public class AnnotationTest {
    @Test
    public void testInsert() throws Exception{
        SqlSessionFactory sqlSessionFactory = new
SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-config.xml"));
        SqlSession sqlSession = sqlSessionFactory.openSession();
        CarMapper mapper = sqlSession.getMapper(CarMapper.class);
        Car car = new Car(null, "1112", "卡罗拉", 30.0, "2000-10-10", "燃油车");
        int count = mapper.insert(car);
        System.out.println("插入了几条记录: " + count);
        sqlSession.commit();
        sqlSession.close();
    }
}
```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.insert - ==>  Preparing: insert into t_car values(null,?,?,?,?,?)
[main] DEBUG lzt.mybatis.mapper.CarMapper.insert - ==> Parameters: 1112(String), 卡罗拉(String), 30.0(Double), 2000-10-10(String), 燃油车(String)
[main] DEBUG lzt.mybatis.mapper.CarMapper.insert - <==    Updates: 1
插入了几条记录：1
```

表配置文件 开始事务 单元格编辑器 筛选 & 排序 列 数据分析 导入 导出 数据

id # bigint	car_num varchar(255)	brand varchar(255)	guide_price # decimal(10,2)	produce_time char(10)	car_type varchar(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源
4	102	丰田未来	40.30	2014-10-05	氢能源
5	102	丰田未来	40.30	2014-10-05	氢能源
30	2001	兰博基尼	100.00	1998-10-11	燃油车
31	2001	兰博基尼	100.00	1998-10-11	燃油车
32	2001	兰博基尼	100.00	1998-10-11	燃油车
33	1112	卡罗拉	30.00	2000-10-10	燃油车

@Delete

- CarMapper接口

```java
import org.apache.ibatis.annotations.Delete;

@Delete("delete from t_car where id = #{id}")
int deleteById(Long id);
```

- AnnotationTest.testDelete

```java
@Test
public void testDelete() throws Exception{
    SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-config.xml"));
    SqlSession sqlSession = sqlSessionFactory.openSession();
    CarMapper mapper = sqlSession.getMapper(CarMapper.class);
    mapper.deleteById(30L);
    sqlSession.commit();
    sqlSession.close();
}
```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.deleteById - ==>  Preparing: delete from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.deleteById - ==> Parameters: 30(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.deleteById - <==    Updates: 1
```

@Update

- CarMapper接口

```
@Update("update t_car set car_num=#{carNum},brand=#{brand},guide_price=#{guidePrice},produce_time=#{produceTime},car_type=#{carType} where id=#{id}")
int update(Car car);
```

- AnnotationTest.testUpdate

```
@Test
public void testUpdate() throws Exception{
    SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-config.xml"));
    SqlSession sqlSession = sqlSessionFactory.openSession();
    CarMapper mapper = sqlSession.getMapper(CarMapper.class);
    Car car = new Car(88L,"1001", "凯美瑞", 30.0,"2000-11-11", "新能源");
    mapper.update(car);
    sqlSession.commit();
    sqlSession.close();
}
```

@Select

- CarMapper接口

```
@Select("select * from t_car where id = #{id}")
@Results({
    @Result(column = "id", property = "id", id = true),
    @Result(column = "car_num", property = "carNum"),
    @Result(column = "brand", property = "brand"),
    @Result(column = "guide_price", property = "guidePrice"),
    @Result(column = "produce_time", property = "produceTime"),
    @Result(column = "car_type", property = "carType")
})
Car selectById(Long id);
```

- AnnotationTest.testSelectById

```
@Test
public void testSelectById() throws Exception{
    SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-config.xml"));
    SqlSession sqlSession = sqlSessionFactory.openSession();
    CarMapper carMapper = sqlSession.getMapper(CarMapper.class);
    Car car = carMapper.selectById(88L);
    System.out.println(car);
}
```